UNIVERSIDAD CATÓLICA ANDRÉS BELLO
Facultad de Ingeniería · Escuela de Ingeniería Informática · Cátedra de Sistemas Operativos

INFORME TÉCNICO: PROYECTO FASE I

Simulador de Kernel y Gestor de Procesos en Consola (Shell & Scheduling Engine)

Integrantes (Grupo 3)	Luis Terrones · Renzo Gallardo · Juan Pablo Rodriguez	**Profesor**	Marcel J. Castro G.
Repositorio GitHub	`github.com/ldt-05/Proyecto-Sistemas-Operativos`	**Fecha de Entrega**	Noviembre de 2026

1. INTRODUCCIÓN

El presente informe técnico documenta el diseño arquitectónico, la estructura modular y el modelo de sincronización del proyecto correspondiente a la Fase I de la asignatura Sistemas Operativos **(Castro, 2026)**. El propósito del sistema es simular el núcleo ejecutor de un sistema operativo de bajo consumo para robótica industrial, integrando una interfaz interactiva de línea de comandos (CLI) de respuesta inmediata con un motor de planificación de CPU (Scheduler) en segundo plano. Siguiendo los fundamentos clásicos de concurrencia y diseño de sistemas operativos **(Silberschatz et al., 2018)**, la solución fue implementada en lenguaje C (estándar C11) empleando hilos POSIX (`pthread`) y cerrojos de exclusión mutua (`pthread_mutex_t`) conforme al estándar IEEE POSIX.1-2017 **(IEEE Computer Society, 2018)**.

2. ARQUITECTURA MODULAR Y DISEÑO DEL SISTEMA

La aplicación se estructuró bajo el principio de separación de responsabilidades, dividiendo las declaraciones de tipos y cabeceras en el directorio `include/` , la implementación lógica en `src/` y el módulo visual opcional en `gui/` , automatizando su compilación con banderas estrictas (`-std=c11 -Wall -Wextra -pthread` , añadiendo `-lmingw32 -mconsole` en Windows) **(Kernighan & Ritchie, 1988)**. Los módulos que conforman el proyecto son:

- **Estructuras del Sistema y Control de Procesos (`include/os_core.h` y `src/main.c`):** Define el bloque de control de procesos (`struct PCB`), el enumerador de estados del ciclo de vida (`NUEVO` , `LISTO` , `EJECUCION` , `BLOQUEADO` y `FINALIZADO`), el arreglo compartido de procesos, las variables de control del planificador y la inicialización/destrucción limpia del cerrojo global `mutex_sistema` .
- **Módulo de Interfaz de Consola — Hilo CLI (`src/cli.c`):** Gestiona el bucle interactivo del prompt `SO-Shell>` , realizando un *parsing* estricto de los comandos ingresados por el operador (`create` , `load` , `ps` , `pcb` , `set_sched` , `set_speed` , `start` , `pause` , `gantt` , `stats` , `gui` , `help` y `exit`). Incorpora validación defensiva de tipos numéricos y cantidad de argumentos para garantizar que la consola jamás sufra bloqueos o cierres abruptos ante entradas malformadas.
- **Módulo del Motor de Scheduling — Hilo CPU (`src/scheduler.c`):** Administra el reloj simulado discreto del procesador mediante la variable acumulativa `clock_ticks` , aplicando un retardo real configurable por ciclo (`set_speed` , por defecto 1000 ms). Ejecuta el despacho de procesos desde la cola de `LISTOS` bajo las políticas **Round Robin (RR)** con quántum configurable y **Shortest Remaining Time First (SRTF)** con expropiación inmediata en caliente.
- **Módulo de Ingesta por Lotes — Batch Processing (`load <archivo>` en `src/cli.c`):** Implementa el analizador léxico para archivos de traza estructurados (`workload_test.txt`), ignorando comentarios (`#`) y

líneas vacías para registrar procesos cuyo ingreso efectivo a la cola de `LISTOS` ocurre exactamente cuando `clock_ticks` iguala su tiempo de llegada (*Arrival Time*).

- **Módulo de Visualización Gráfica — Bonus en Tiempo Real (`gui/` con Raylib):** Implementado en `gui/gui_main.c`, `gui/gui.c`, `gui/gui_data.c`, `gui/gui_gantt.c` y `gui/gui_style.c`. Despliega una ventana interactiva con `raylib` (30 FPS) para monitorear en vivo el reloj, los contadores de estado, la tabla de PCBs y el Diagrama de Gantt mientras `HiloCLI` y `HiloMotor` continúan operando concurrentemente.

Métrica del PCB / Sistema	Expresión Matemática (en Ticks)	Comportamiento en Simulación
Tiempo de Retorno (TAT)	`TAT = T_Finalizacion - T_Llegada`	Se calcula al pasar a estado `FINALIZADO`; muestra `N/A (En ejecución)` mientras continúa activo.
Tiempo de Espera (WT)	`WT = TAT - Burst_Inicial`	Incrementa en +1 por cada tick discreto en que el proceso permanece esperando en estado `LISTO`.
Tiempo de Respuesta (RT)	`RT = T_1ra_Ejecucion - T_Llegada`	Se fija en el instante exacto en que la CPU despacha al proceso por primera vez.

3. DIAGRAMA DE HILOS Y SINCRONIZACIÓN

El núcleo opera con dos hilos POSIX concurrentes creados desde `main()` mediante `pthread_create()` (más el hilo principal de renderizado a 30 FPS cuando se ejecuta en modo gráfico con `raylib`). Dado que los hilos leen y modifican concurrentemente la tabla de PCBs y el estado del reloj, todas las secciones críticas están protegidas por el cerrojo `mutex_sistema` **(IEEE Computer Society, 2018)**:



Hilo Concurrente	Operaciones en Sección Crítica	Prevención de Condiciones de Carrera y Deadlocks
Hilo CLI (Interfaz de Usuario)	Inserción de nuevos PCBs (`create` / `load`), lectura instantánea de estados (`ps` , `pcb` , `gantt` , `stats`) y cambio de algoritmo en caliente (`set_sched`).	Adquiere `mutex_sistema` únicamente durante la copia o actualización de memoria y lo libera antes de esperar nueva entrada por `fgets()` .
Hilo Motor CPU (Planificador)	Evaluación de llegada de procesos, selección de ráfaga mínima (SRTF) o rotación de quántum (RR), descuento de `burst_restante` y registro de Gantt.	Bloquea `mutex_sistema` al procesar el tick actual y lo libera inmediatamente antes de invocar el retardo real (`usleep`), evitando congelar la CLI.

4. GUÍA DE COMPILACIÓN, EJECUCIÓN E INTERFAZ GRÁFICA

El repositorio incluye compilación automatizada mediante `Makefile` (para Linux, macOS y WSL) así como compilación directa con `gcc` en Windows (MSYS2 UCRT64). A continuación se detallan los pasos exactos para preparar el entorno, compilar y abrir la interfaz gráfica en cualquier equipo:

4.1. Clonar el repositorio e instalar dependencias

Para el **modo consola obligatorio** únicamente se requiere un compilador compatible con C11 y soporte POSIX Threads (`gcc` y `make`). Para habilitar el **bonus de interfaz gráfica en tiempo real**, debe instalarse la biblioteca `raylib` y `pkg-config` según el sistema operativo:

Sistema Operativo	Comando de instalación de dependencias (GCC + Raylib)
Linux / Ubuntu / WSL*	`sudo apt update && sudo apt install -y build-essential pkg-config libraylib-dev`
macOS (Homebrew)	`brew install raylib pkg-config`
Windows (MSYS2 UCRT64)	`pacman -S mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-raylib pkg-config`

**Nota para WSL:* la ventana gráfica requiere Windows 11 con soporte WSLg o un servidor X11; si se ejecuta en consola pura sin entorno gráfico, el modo CLI funciona de forma autónoma sin instalar `raylib` .

4.2. Compilar e iniciar con Interfaz Gráfica (Raylib + CLI)

Ubíquese en la raíz del proyecto descargado (`Proyecto-Sistemas-Operativos/`) y ejecute según su plataforma:

- **En Linux, macOS o WSL (usando `Makefile`):**

```
git clone https://github.com/ldt-05/Proyecto-Sistemas-Operativos.git
cd Proyecto-Sistemas-Operativos
make gui
./gui/simulador_gui
```

- **En Windows (PowerShell / MSYS2 UCRT64):**

```
gcc gui/gui_main.c gui/gui.c gui/gui_style.c gui/gui_data.c gui/gui_gantt.c src/cli.c src/scheduler.c -o gui/simulador_gui.exe -std=c11 -O2 -Wall -Wextra -pthread -IC:/msys64/ucrt64/include -LC:/msys64/ucrt64/lib -lraylib -lm
.\gui\simulador_gui.exe
```

(Si ejecuta desde MSYS2 con `make` instalado, también puede compilar con `make -f gui/Makefile` y lanzar `./gui/simulador_gui`).

4.3. Cómo iniciar en Modo Consola Pura y Pruebas Automatizadas

```
# Linux / macOS / WSL (usando Makefile):
make && ./simulador_so       # Inicia el simulador en modo consola pura
make test                     # Ejecuta las 9 pruebas automatizadas del motor (tests/test_motor.c)

# Windows (compilación directa sin Makefile):
gcc src/main.c src/cli.c src/scheduler.c -o simulador_so.exe -std=c11 -Wall -Wextra -pthread -lmingw32 -
mconsole
.\simulador_so.exe
```

4.4. Operación de la Interfaz Gráfica y Prueba Rápida de Validación

> **Arquitectura de doble ventana:** al abrir `simulador_gui` se despliega la ventana gráfica *"Simulador SO - Monitor de Procesos"* y, simultáneamente, queda activo el prompt interactivo `SO-Shell>` en la terminal de origen. La ventana gráfica actúa como tablero de monitoreo en vivo (*solo lectura*): el reloj inicia pausado en `TICK 0` y todos los comandos se introducen desde la terminal mientras la interfaz visual se actualiza al instante.

- **Paneles visibles en la GUI:** encabezado con contador `TICK` y botón `Cerrar GUI` (cierra solo la ventana manteniendo viva la terminal); 5 tarjetas resumen (`TOTAL`, `NEW`, `READY`, `RUNNING`, `FINISHED`); tabla de PCBs con desplazamiento vertical mediante la rueda del mouse; y Diagrama de Gantt en vivo de los últimos 32 ticks (muestra el nombre/PID de cada proceso y detalle completo al pasar el cursor).
- **Secuencia recomendada para validar el Escenario 2 en vivo desde `SO-Shell>`:** ejecutar en orden `set_sched SRTF` → `load workload_test.txt` → `start`. En pantalla se observará cómo `Job_A` inicia en `t=0`, es expropiado por `Job_B` en `t=1`, continúa `Job_C` en `t=3` y retoma `Job_A` de `t=4` a `t=7`. Al terminar, consultar `gantt` y `stats` y escribir `exit` para cerrar limpiamente ambos hilos y la ventana.

5. GESTIÓN DEL PROYECTO EN GITHUB (METODOLOGÍA KANBAN)

El control de versiones y el trabajo colaborativo se gestionaron en el repositorio oficial del equipo (`https://github.com/ldt-05/Proyecto-Sistemas-Operativos`) aplicando la metodología ágil Kanban a través de **GitHub Projects**. El tablero se estructuró en las columnas reglamentarias *To Do*, *In Progress* y *Done*, dividiendo el desarrollo mediante *Issues* etiquetados por módulo y ramas de características (*feature branches*) integradas mediante *Pull Requests*:

- **Luis Terrones (Arquitectura Base e Interfaz CLI):** Definición de estructuras en `os_core.h`, inicialización de hilos y mutex en `main.c`, validación de sintaxis en `cli.c` e implementación de comandos de creación y consulta (`create`, `ps`, `pcb`, `help`, `exit`).
- **Renzo Gallardo (Motor de Ejecución y Planificación):** Implementación del reloj discreto y control de velocidad en `scheduler.c`, algoritmos apropiativos Round Robin y SRTF, registro de intervalos del diagrama de Gantt y cálculo de estadísticas globales (`start`, `pause`, `set_sched`, `set_speed`, `gantt`, `stats`).
- **Juan Pablo Rodriguez (Procesamiento por Lotes, Pruebas e Interfaz / Documentación):** Desarrollo del parser de carga masiva (`load <archivo>`), diseño de baterías de prueba (`workload_test.txt`), módulo de visualización gráfica en Raylib (`gui/`), administración del tablero Kanban en GitHub y elaboración del Informe Técnico.

6. ANEXO DE INTELIGENCIA ARTIFICIAL (DECLARACIÓN OBLIGATORIA)

> **Declaración de Uso Ético (Sección 9 de la Rúbrica):** El equipo declara que las herramientas de Inteligencia Artificial se emplearon estrictamente bajo el rol de **tutor socrático, consultor conceptual de arquitectura y asistente de depuración (debugging), sin delegar la autoría ni la comprensión lógica del código fuente.**

- **Herramientas utilizadas:**
  - **Claude (Anthropic) / Antigravity:** Utilizados como asistentes de depuración para auditar el alcance de los bloqueos con `pthread_mutex_lock`, prevenir condiciones de carrera entre el Hilo CLI y el Hilo Motor, y verificar las banderas de compilación POSIX en MinGW-w64.
  - **Google Gemini:** Utilizado en modalidad de tutor socrático para validar la lógica de expropiación del algoritmo SRTF, diseñar pruebas de casos borde en el parser de la consola y estructurar la revisión formal de este informe técnico.
- **Prompts principales utilizados:**
  1. *"Actúa como tutor socrático en Sistemas Operativos. Sin darme el código resuelto, guíame mediante preguntas para identificar qué variables globales de la estructura PCB requieren protección con pthread_mutex_lock cuando el Hilo CLI consulta el comando ps mientras el Hilo Motor actualiza las ráfagas".*
  2. *"Analiza esta función de lectura interactiva en C con fgets y strtok: ¿qué entradas malformadas del usuario podrían ocasionar un fallo de segmentación o bloqueo en la CLI y qué validaciones defensivas debemos programar?".*
  3. *"Necesito auditar nuestra lógica de desapropiación en SRTF para verificar por qué en t=1 un proceso recién llegado con menor ráfaga debe desplazar inmediatamente al proceso en ejecución y cómo consolidar los intervalos continuos en el comando gantt".*
- **Validación y gestión crítica de resultados:** Todas las sugerencias diagnósticas fueron verificadas manualmente por los tres integrantes del grupo. Se ejecutaron los dos escenarios oficiales de validación de la cátedra: (1) prueba interactiva en caliente alternando de `RR` (quántum 2) a `SRTF` e introduciendo comandos erróneos (`createProceso_A`, `pcb abc`, `set_sched FIFO`) comprobando cero colapsos; y (2) prueba por lotes con `workload_test.txt` bajo `SRTF`, verificando que la salida de `gantt` produjera exactamente la secuencia teórica esperada: `[0-1s: Job_A] -> [1-3s: Job_B] -> [3-4s: Job_C] -> [4-7s: Job_A]`.

7. REFERENCIAS BIBLIOGRÁFICAS

Castro, M. J. (2026). *Especificaciones y Rúbrica del Proyecto — Fase I: Simulador de Kernel y Gestor de Procesos en Consola (Shell & Scheduling Engine)*. Cátedra de Sistemas Operativos, Universidad Católica Andrés Bello (UCAB).

IEEE Computer Society. (2018). *IEEE Std 1003.1-2017 (POSIX.1-2017): Standard for Information Technology—Portable Operating System Interface (POSIX®)*. Institute of Electrical and Electronics Engineers.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.